

Neurobridge: The Companion App for Brain-Computer-Interfaces

Healthcare Innovation Program: Development Report for Digital
Healthcare Solutions

Author: Tobias Wen Klingenberg
Matriculation Number: ---
School: TUM School of Computation, Information and
Technology
Date: 13.06.2026

Table of Contents

List of Abbreviations	ii
1 Introduction	1
1.1 Background: Brain-Computer Interfaces and Digital Health	1
1.2 Motivation and Background	2
1.3 Problem Statement and Objectives	2
1.4 Structure of this Report	2
2 Requirements and Target Users	3
2.1 Stakeholders and Use Context	3
2.2 Functional Requirements	3
2.3 Non-Functional Requirements	4
2.4 Accessibility Requirements for Motor-Impaired Users	5
3 System Architecture	6
3.1 Technology Background	6
3.2 Architectural Overview	6
3.3 Data Model and Backend	7
3.4 Authentication and Role Separation	9
3.5 Client Applications	10
3.6 Deployment and Infrastructure	11
4 Regulatory and Data Protection Compliance	13
4.1 European Legal Conditions for Digital Health	13
4.2 Data Protection by Design	13
4.3 Platform Choice: Supabase as a Self-Hostable First Step	14
4.4 Compliance Limitations of the Current Prototype	14
5 Implementation Challenges and Design Decisions	15
5.1 Cross-Platform Code Sharing	15
5.2 Designing for Accessibility: Motor-Impaired and Assistive-Technology Users	15
5.3 Offline Resilience and Data Integrity	15
5.4 Prototype Boundaries: Mocked and Deferred Components	16
6 Conclusion	16
6.1 Summary	16
6.2 Limitations	17
6.3 Outlook	17
References	19

List of Abbreviations

API	Application Programming Interface
APK	Android Package (Android app build artifact)
BCI	Brain-Computer-Interface
CSV	Comma-separated Values
EAS	Expo Application Services
EEA	European Economic Area
EU	European Union
GDPR	General Data Protection Regulation
IPA	iOS App Package (iOS app build artifact)
JSON	JavaScript Object Notation
MDR	Medical Device Regulation
REST	Representational State Transfer
RLS	Row-Level Security
UI	User Interface
URL	Uniform Resource Locator
UTC	Coordinated Universal Time
WCAG	Web Content Accessibility Guidelines

1 Introduction

Neurobridge is a two-part software system designed to support the ongoing care of individuals who have recently received a Brain-Computer-Interface (BCI) implant (Hochberg et al., 2012). The system integrates a mobile application that participants use in their daily lives with a web portal accessed by clinical staff overseeing the implant program. The mobile application facilitates short daily check-ins regarding participants' conditions, maintains records of lab appointments, enables communication with both the laboratory and other participants, and provides home training opportunities. The web portal allows the research team to review collected information, document lab session outcomes, manage participant accounts, and communicate with participants. Consequently, the system serves as a shared record between participants and the clinical team responsible for their recovery. Digital monitoring is particularly important for this group, as the post-implantation period is when subtle changes are most likely to be overlooked, yet are most critical to detect. For example, participants may experience poor sleep, miss physiotherapy sessions, or observe early signs of infection well before their next scheduled visit. When such information is recorded on paper, communicated via phone calls, or remembered by staff, it is transmitted slowly and is challenging to analyze for trends. Consolidating this data in a structured digital platform and enabling future home training tackles these challenges, an approach consistent with evidence that mobile-health interventions can support monitoring and adherence (Free et al., 2013).

1.1 Background: Brain-Computer Interfaces and Digital Health

A Brain-Computer Interface (BCI) is a device that records electrical activity from the brain and translates it into commands for a computer or another device. For a person with severe paralysis, such as tetraplegia following a spinal-cord injury, an implanted BCI can restore the ability to control the body no longer provided, for instance, moving a cursor, typing, or operating a robotic arm by intention alone (Hochberg et al., 2012; Wolpaw et al., 2002). Such an implant does not work immediately. A decoder has to learn how each person's neural signals map to intended actions, and that mapping is built up and refined over repeated training sessions in the lab, where the participant performs guided tasks while the system measures how accurately and how quickly the intended target is reached. Adaptation and recovery, therefore, unfold over months rather than in a single procedure, and the quality of that process depends on consistent follow-up between visits.

Digital health is the use of software and connected devices to support care outside the clinic. Mobile and web applications can gather structured self-reports, present trends to clinicians, and keep the participant and the care team in contact without relying on paper forms or phone calls, and such interventions have been associated with

improved monitoring and adherence in long-term care (Free et al., 2013). Neurobridge applies this approach to the particular setting of post-implant BCI rehabilitation.

1.2 Motivation and Background

Post-implantation care for BCI recipients is a continuous process. Participants are required to attend regular laboratory visits, stick to physiotherapy schedules, and remain observant for warning signs, while clinical teams require consistent data on physical recovery, sleep patterns, symptoms, and interface performance during each session (Wolpaw et al., 2002). Currently, these data streams are often fragmented across different, non-integrated tools, such as paper questionnaires completed at home, laboratory-maintained spreadsheets, and ad-hoc communications when issues arise. This fragmentation not only causes inconvenience but also impedes adherence monitoring, delays the communication of critical reports to clinicians, and prevents reliable auditing of data entries. The design of Neurobridge is informed by two key observations: first, the distinct needs and working conditions of participants and clinical staff necessitate separate interfaces to avoid compromising usability for either group; second, the inclusion of individuals with severe motor impairments means that the ease of daily reporting is essential for data collection. These considerations shaped the system requirements prior to any technological decisions and are reflected throughout the report. Furthermore, there is currently no implementation that enables complete testing sessions outside the laboratory setting. As BCI implants transition from research projects to broader clinical use (Collinger et al., 2013), the existing approach is unsustainable, underscoring the need for digital follow-up and in-home training solutions.

1.3 Problem Statement and Objectives

This report documents the design and implementation of Neurobridge as a first demonstrable prototype. It combines a participant-facing mobile application and a lab-facing web portal built on a shared Postgres and Supabase backend, with a single data-access layer serving both clients. There were two initial objectives. The first goal was a working system, accessible and compliant by design, that a clinical stakeholder could use end-to-end and evaluate. The second was to precisely establish the gap between this prototype and a system that could safely process real patient data at scale.

1.4 Structure of this Report

This report is structured to progress from requirements to system architecture, then to compliance considerations, and conclude with a reflective conclusion. Chapter 2 outlines the requirements and describes the two primary user groups, with particular attention to accessibility for motor-impaired users. Chapter 3 details the system architecture. Chapter 4 examines regulatory and data protection compliance as it relates to the system's architecture. Chapter 5 discusses implementation challenges

and the trade-offs encountered. Chapter 6 concludes with a summary of limitations and provides an outlook for future development.

2 Requirements and Target Users

Before any architectural choice was made, the project fixed who the system was for and what those users needed to accomplish. Neurobridge serves two clearly separated audiences whose working conditions differ sharply, and this separation drove the principal design decisions: the shape of the data model, the decision to build two clients rather than one, and the emphasis on accessibility for participants. The chapter presents the stakeholders and their context, derives the functional and non-functional requirements from the implemented system, and treats accessibility as a primary requirement in a dedicated section.

2.1 Stakeholders and Use Context

There are two primary roles, and both authenticate against the same Supabase project. Participants are people living with a recent BCI implant who use the mobile application on their own phones. Lab and clinical staff are the researchers and clinicians who run the implant program and use the web portal. A single account belongs to exactly one role, and that role decides both which application a person may use and which data they may see. The two groups use the software under very different conditions. A participant uses the app daily and unsupervised at home, frequently, while fatigued, and sometimes with adapted input hardware such as a mouth stick or head pointer. Each interaction must remain feasible for someone with limited or no hand use, and we cannot assume a helper is present, which may assist the impaired user. Lab staff, by contrast, use the portal at a desk during or shortly after a scheduled visit. Their task is data-intensive: they read a participant's history and enter structured measurements from a training session. Speed and density matter more than simplicity there, whereas on the participant side, simplicity and tolerance of imprecise input matter more than density. Designing a single interface that satisfied both would have meant compromising each, which is the central reason the system has two distinct front ends rather than a single shared backend.

2.2 Functional Requirements

The functional requirements below are derived from the implemented feature set and from requirements provided by actual test users during development.

Participant application.

- **FR1 (Daily check-in).** A participant can submit one structured daily report covering physiotherapy attendance and type, sleep quality and duration, whether they feel unwell and with which symptoms, and an optional free-text note to the lab. The flow is a short sequence of single-question steps.

- **FR2 (Diary).** A participant can keep an append-only personal journal with several entries per day, and can delete an entry they wrote.
- **FR3 (Calendar).** A participant can browse a calendar that marks days with a scheduled lab visit, a diary entry, or both, and can open a read-only overview of any single day.
- **FR4 (Visit history and progress).** A participant can review past and upcoming lab visits and see aggregated training metrics, including a cumulative view of active training minutes over time.
- **FR5 (Training exercises).** A participant can work through a sequence of practice mini-games whose progress persists between app launches (As Demo).
- **FR6 (Messaging).** A participant can hold a private conversation with the lab team in a one-to-one message thread, and can take part in a shared community room where participants talk with one another.

Lab portal.

- **FR7 (Participant list).** A lab member can see all participants and open a detail view for any one of them.
- **FR8 (Per-participant review and reply).** Within that detail view, a lab member can read the participant's check-ins, visit history, and derived performance data, and can read and reply to that participant's message thread.
- **FR9 (Record lab visits).** A lab member can create and edit a lab visit, including its session metrics, which are stored as a typed CSV object. A visit can be populated from an uploaded session file.
- **FR10 (Account management).** A lab member can create and manage participant and lab accounts.

Shared.

- **FR11 (Authentication).** Every user signs in with an email and password, and a user is only admitted to the application that matches their role.
- **FR12 (Role-based access).** Each role sees only the data it is entitled to. A participant sees their own records only, a lab member sees everyone's.

2.3 Non-Functional Requirements

Even at the prototype stage, a healthcare-adjacent tool has obligations beyond its feature list. Four non-functional requirements most directly shaped the design.

Data integrity. Clinical records must not be silently corrupted by a misbehaving or retried client. The clearest example is the rule that a participant may submit at most one check-in per day. Because this is a correctness property of the record, it is enforced in the database rather than only in the form. Similarly, questionnaire history is append-only, so an earlier report cannot be altered retrospectively.

Availability for a vulnerable group. A participant should not lose a report or be blocked from seeing their own information because the network was slow when the app started. The app is therefore expected to render its last-known data without a live

connection, so that a temporary outage degrades the experience rather than rendering it unusable.

Low cognitive load. Because participants may be tired, unwell, or using adapted input, each task should require as few decisions and as few taps as possible, and the path to any screen should be predictable. This requirement is what motivates the single-entry-point navigation described below.

Auditability. Clinical session data is entered by staff, and it should be possible to attribute each record to the person who created it. Lab visits, therefore, carry their author and are timestamped, providing a basic audit trail of who recorded a session and when.

2.4 Accessibility Requirements for Motor-Impaired Users

Accessibility is identified here as a separate requirement category because, for this population, it is a precondition for the system to function at all. This follows the principle of ability-based design, which holds that a system should adapt to a user's abilities rather than require the user to conform to it (Wobbrock et al., 2011). The participant group includes people with tetraplegia who operate a touchscreen through a mouth stick, a head pointer, or a similar adapted pointer, and who may have little or no use of their hands. For such a user, a screen that assumes precise fingertip control, a steady hand, or two-handed gestures is unusable. From this, the project set the following requirements for the participant app. Touch targets must be large and widely spaced, so that an imprecise pointer still lands on the intended control point. Interaction must tolerate imprecision and there can not be reliance on fine drag gestures, on multitouch, or on hover-only affordances, because an assistive pointer typically produces a single tap. Each task must be reachable in as few steps as possible. Information must never be carried by color alone, and type must stay legible. Finally, navigation must be predictable and offer a single obvious entry point. This last requirement is a direct reason the participant app is built around a single Home screen with large tiles rather than a crowded bottom tab bar: fewer simultaneous targets mean fewer opportunities to mis-tap. WCAG 2.2 (World Wide Web Consortium, 2023) serves as the guideline for these goals. The most relevant success criteria to mouth-stick and switch-style input are Target Size, which influences directly to how large and well-spaced controls must be. Pointer Gestures and Dragging Movements argue against path-based or multi-point gestures that such users cannot perform, and the general principle that no task should depend on fine motor precision. This is a design objective, not a certified conformance result. The system was designed to meet these criteria and was assessed against them only informally.

3 System Architecture

Neurobridge is organized as a single pnpm monorepo containing two client applications and one shared library, all built on a single Supabase backend. The mobile application for participants is built with Expo and React Native, while the web portal for lab staff is built with Vite and React. Both access the data through a shared TypeScript package rather than communicating with the backend directly. The backend itself is a Supabase project providing Postgres, authentication, Row-Level Security, and Edge Functions.

3.1 Technology Background

The system is created from a small number of widely used components, summarised here for readers unfamiliar with them. [Supabase](#) is an open-source backend platform that combines a [PostgreSQL](#) database (a mature relational database that stores data in tables), user authentication, and Edge Functions, which are small pieces of server-side code that run on demand. Row-Level Security is a PostgreSQL feature that attaches access rules to individual rows of a table, so the database itself decides which rows a given user may read or write. On the client side, the participant app is built with [React](#) Native and Expo, a framework and toolchain for writing a single codebase that runs as a native app on both iOS and Android, while the lab portal is a web application built with React and Vite, a fast build tool for the browser. TanStack Query is a library for retrieving data from the backend and caching it on the device, so that recently viewed data remains available offline. The project is organized as a monorepo, a single code repository that contains both applications and the code they share.

3.2 Architectural Overview

The system decomposes into four pieces. `apps/mobile` is the participant app and `apps/web` is the lab portal. `packages/shared`, published internally as `@bci/shared`, owns everything that uses the database. A Supabase client factory, the generated database types, domain types reshaped for application code, and a set of TanStack Query hooks, that wrap every read and write. The last piece is the Supabase project, which contains the schema, security policies, and server-side functions. The rule that binds these together is that all database access goes through `@bci/shared`. Neither app calls the Supabase REST endpoint with a raw request, nor does either app construct a Supabase client on its own. Both obtain one from the shared factory and issue every query through a shared hook. The result is consistency at a single point of change. When a column is renamed or a new metric is added, the type definitions and hooks are updated once in the shared package, and both clients see the change through their compilers rather than failing silently at runtime in whichever app was forgotten.

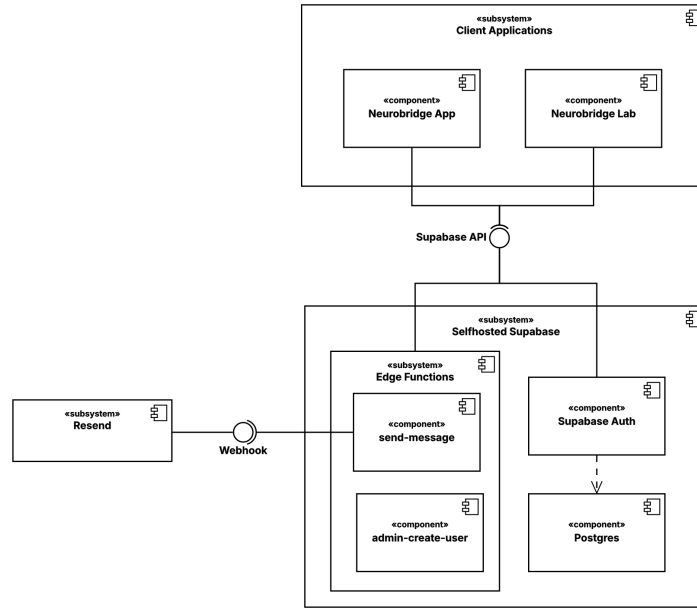


Figure 1: Component diagram. The mobile and web clients reach the Supabase backend (Auth, Postgres with Row-Level Security, Edge Functions) only through the shared package. Resend is provided via a send-message Edge Function.

3.3 Data Model and Backend

The backend is a Postgres database, of which the schema encodes the domain directly. The `profiles` table holds one row per authenticated user, carrying the user's role, name, and, for participants, implant details. Crucially, a profile is not created by application code. A trigger on `auth.users` inserts the matching profile when an account is created, taking the role and name from the signup metadata. This removes a gap that would otherwise exist. There is no moment when a user account exists without a corresponding profile, so no client is responsible for creating one, and no code path has to handle the absence of a profile.

The `daily_questionnaires` table stores participant check-ins and is append-only. The one-per-participant-per-day rule from the requirements is enforced by a unique constraint on the participant together with a stored, generated column holding the UTC date of submission, rather than by client-side validation. The generated column is used because truncating a timestamp to a day is not an immutable operation in Postgres and so cannot be indexed directly. Adding the date into a stored column instead makes the uniqueness constraint possible. Therefore, the correctness does not depend on the app's behavior. Even a duplicated or retried submission is rejected by the database.

The `lab_visits` table treats a visit as a training session: the two are the same event in this domain, so the session's measurements reside on the visit row. Those measurements are stored in a single `results` column typed as `jsonb`, whose shape is described by the `VisitResults` interface in the shared package. Keeping the metrics in a typed

JSON column rather than in a wide set of scalar columns lets the measurement set evolve, and it keeps online-only figures such as success rate and time-to-target naturally optional for offline sessions. Two further tables complete the original model: `diary_entries` holds the participant's personal journal with multiple entries per day, and `lab_visit_attendees` records which lab members attended a given visit as a many-to-many relation. A later addition introduced messaging through two more tables. `chat_messages` is a private thread between a participant and the lab team, keyed by the participant who owns it. Because the schema has no fixed participant-to-clinician assignment, any lab member can read and reply to any thread, whereas a participant sees only their own. `community_messages` is a single shared room in which participants talk with one another, readable by every signed-in user but writable only by participants. Both tables follow the same Row-Level Security discipline as the rest of the schema, so which thread a person may read and where they may post is decided in the database rather than in the clients.

A recurring design principle throughout the schema is that invariants should be enforced at the database level. Requirements such as limiting users to one check-in per day, associating every visit with an author, ensuring that each user has a corresponding profile, and maintaining an append-only history are implemented through constraints and triggers rather than relying on client-side implementation checks. For a system handling health relevant data, this approach represents a deliberate reliability decision. The integrity guarantees remain consistent regardless of which client application performs the write operation and are preserved even in the presence of client-side errors or defects.

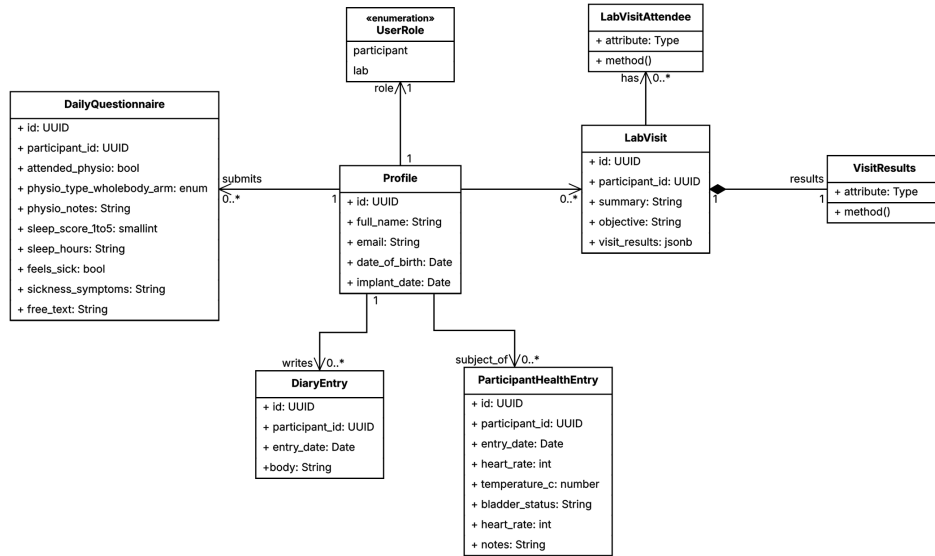


Figure 2: Class diagram of the domain model, following the Postgres schema and the reshaped domain types in @bci/shared. VisitResults is the typed shape of the lab_visits.results JSONB column, modeled as a value object composed into LabVisit.

3.4 Authentication and Role Separation

Access control is layered, and only the lower layer is a security boundary. Row-Level Security in Postgres is the boundary. Every table has RLS enabled, and the policies are written so that a participant can read and write only their own rows, while a lab member can read everything and write the records their role owns. A participant cannot retrieve another participant's data, regardless of what the client sends, because the database withholds the rows. The policies are kept readable by routing the role lookup through a SECURITY DEFINER helper, `user_role()`, which reads the requester's role without recursing back through RLS on the profiles table. The role itself is fixed at signup and then immutable through the ordinary API. It is written once from the signup metadata by the profile-creation trigger, and a second trigger rejects any later attempt to change it, so a participant cannot promote their own account. Only a service role connection, which bypasses RLS, can alter a role, and that path is reserved for lab-side administration. Layered on top of this, and explicitly not a security control, is an app-side role check. After a successful sign-in, each client fetches the caller's profile and, if the role does not match the application (e.g., a lab user opening the participant app, or the reverse), signs the session out with a clear message. This belongs to the user experience: it directs users to the appropriate application.

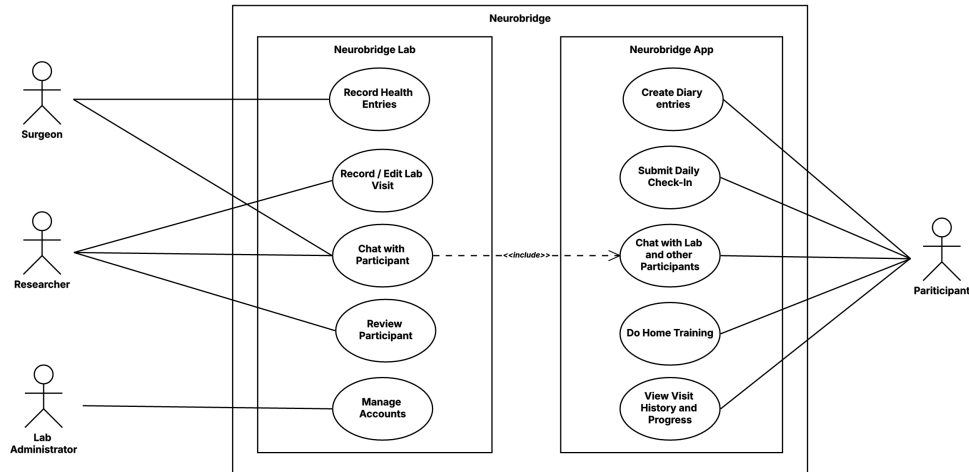


Figure 3: Use case diagram. The participant and lab-staff actors and the use cases each role can reach. The split is enforced by Row-Level Security in the database, not only by the interface.

3.5 Client Applications

The two front ends share their data layer, and nothing else, and that is intentional. The participant app uses Expo Router for file-based navigation and NativeWind for styling. Its navigation is built around a single Home screen of large tiles, with other destinations reached by tapping a tile rather than through a permanent tab bar, which serves the accessibility requirements from the previous chapter. It is offline-tolerant: its TanStack Query cache is persisted to device storage so that the last-known check-ins and visits render even when the app starts without a connection. The web portal uses Vite, React Router, and the shadcn/ui component set, and is laid out as two panes: a participant list on the left and a routed detail panel on the right, with the selected participant encoded in the URL so a view can be linked and shared.

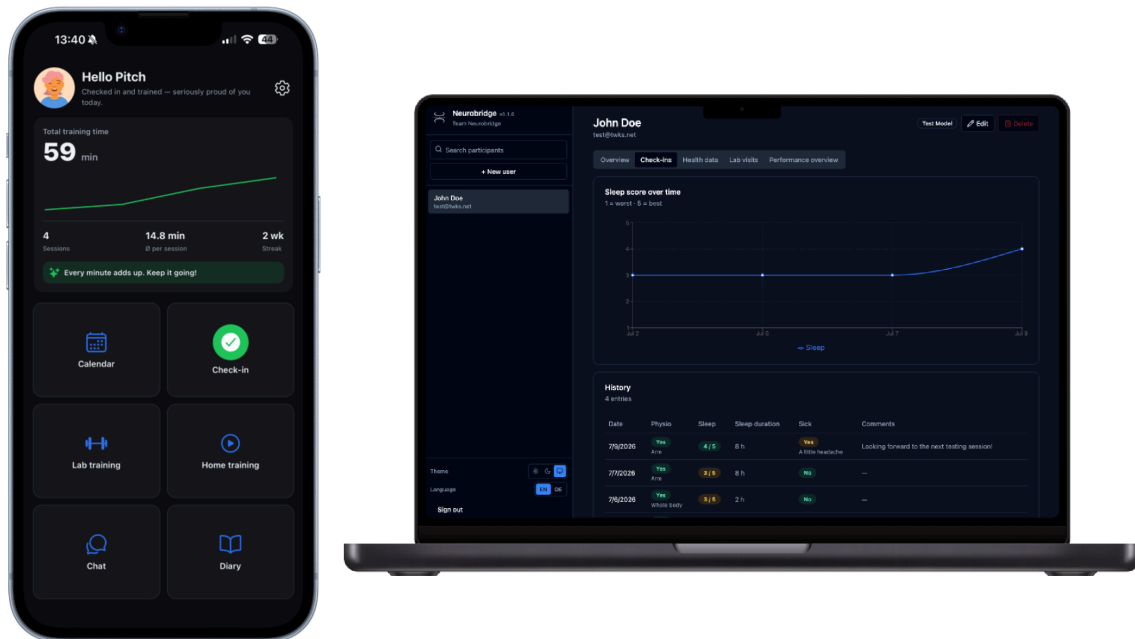


Figure 4: UI of the Participant App (left) and the Lab Portal (right).

The design principle is that each app owns its own UI primitives and routing, and that only data access and types are shared. React Native and the web render through different component models and lay out differently, and their routers use different conventions. Forcing them behind a single shared UI abstraction would have produced a compromise layer that suited neither platform well, and it would have cost more to maintain than the limited duplication it would have avoided.

3.6 Deployment and Infrastructure

The pieces run in a lightweight configuration suited to a prototype. During development, the whole backend runs locally as a Dockerized Supabase stack, reset from migrations and a seed file, so a developer has a full database, auth service, and functions on one machine. For shared and demonstration use, the same schema runs on a hosted Supabase project; the web portal is a static Vite build; and the mobile app is distributed via Expo and EAS builds (an APK for Android and an IPA for iOS) rather than through the public app stores at this stage. This is deliberately a demonstration-grade deployment, chosen so that effort was directed at the product rather than at operational infrastructure, which a prototype does not yet require. At present, both the lab portal and the participant app are hosted on a self-hosted machine reachable over the internet. The Compliance chapter sets out what this deployment would have to become, in particular, where the data is hosted and under what agreements, before the system could process real patient data.

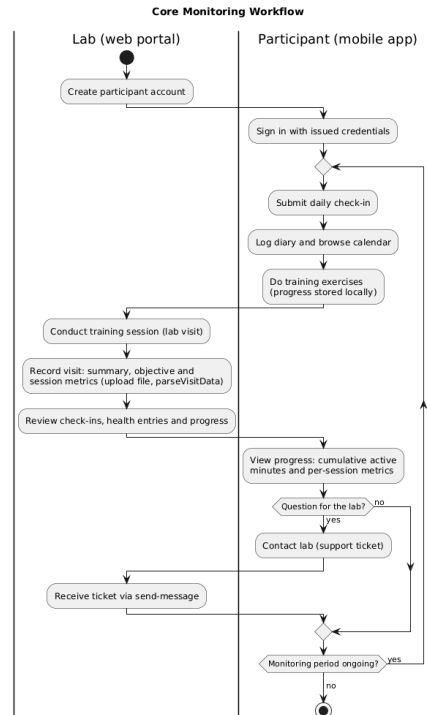


Figure 5: Activity diagram of the core monitoring workflow. The lab provides the participant with training sessions, while the participant reports and reviews progress through the app. What the lab records feed back into the participant's progress, and the cycle repeats for the duration of the monitoring period.

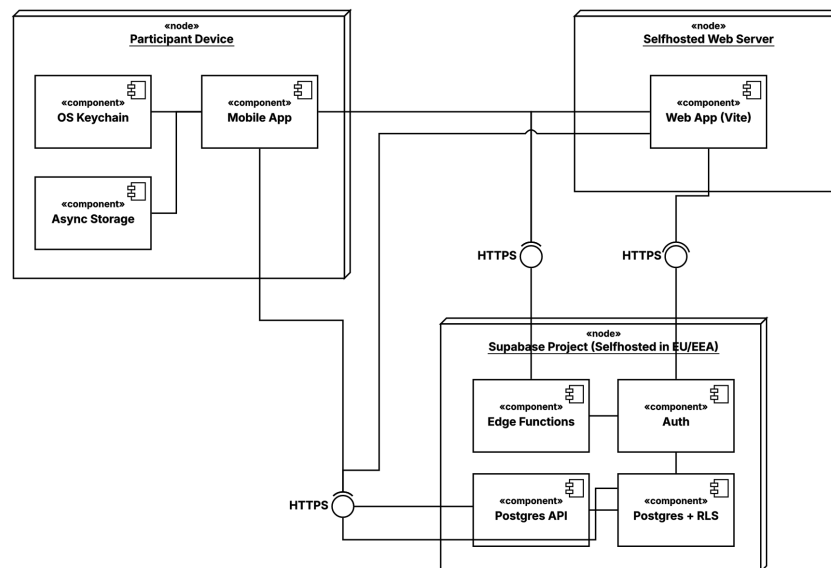


Figure 6: Deployment diagram. Where each build artifact runs (participant device, lab-staff browser, and the Supabase project as hosted or self-hosted infrastructure) and the services each client talks to.

4 Regulatory and Data Protection Compliance

Daily check-ins, symptom reports, and BCI session metrics are health data, which is a special category under European law. For this reason, compliance must inform the architecture from the outset rather than be addressed only after the system is complete, because much of what the law requires concerns how data is stored and who can access it. This chapter reviews the relevant legal conditions, maps the architecture of the previous chapter onto data protection principles, justifies the choice of Supabase at this stage, and identifies what the prototype does not yet solve.

4.1 European Legal Conditions for Digital Health

Three frameworks bear on a system of this kind. Two apply directly, and one does not. The General Data Protection Regulation, Regulation (EU) 2016/679 (European Parliament and Council of the European Union, 2016), is the baseline. Its Article 9 treats data concerning health as a special category whose processing is prohibited unless a specific condition is met, and it requires heightened safeguards around such data. Everything Neurobridge stores about a participant falls under this heading, so the system requires both a lawful basis for processing and technical measures proportionate to the data's sensitivity.

The European Health Data Space Regulation, Regulation (EU) 2025/327 (European Parliament and Council of the European Union, 2025), is the more recent and more health-specific framework. It governs the primary use of electronic health data, meaning its use in providing care, and its secondary use for purposes such as research, and it sets expectations for interoperability and for patient control over health records across the Union. A production version of a system like this would fall within its scope, which is why it is included here even though the prototype does not yet meet its interoperability obligations.

The EU Medical Device Regulation, Regulation (EU) 2017/745 (European Parliament and Council of the European Union, 2017), is deliberately out of scope at present, and this should be stated explicitly in order to characterize the system accurately. Neurobridge is a monitoring and adherence tool, it records a participant's condition and the outcome of each session. It does not diagnose, drive the implant, or make or inform an automated clinical decision. This places it below the threshold at which software is classified as a regulated medical device. That threshold must be monitored rather than assumed, because it can shift as the functionality grows, and the Outlook returns to it.

4.2 Data Protection by Design

The architecture already implements several data-protection principles in technical form. Row-Level Security is a clear example. Data minimization and access control under the GDPR require that each actor can reach only the data they need. In Neurobridge, the database enforces this directly. A participant's session can read

that participant's rows and no others, and a lab member's access is scoped to their role, with the database itself refusing anything outside those bounds. Because the boundary is enforced below the application, a client bug cannot widen it. The immutable role column is a second, concrete instance of privacy by design. A participant cannot escalate their own access because the field that determines access is set once at account creation and then locked from change through the ordinary interface. Privilege, therefore, cannot drift upward through the front end. Row-Level Security does not address every requirement on its own. It controls who may read or write a row, but it does not, by itself, record who read what and when, so there is no application-level audit trail of access. It does not manage consent, and it does not encode a retention or erasure schedule that would satisfy the GDPR's storage-limitation and right-to-erasure requirements. These capabilities must be supplied by a separate mechanism. They fall outside the scope of the current prototype, and the Limitations section addresses them.

4.3 Platform Choice: Supabase as a Self-Hostable First Step

The choice of Supabase for this stage rests on three considerations. First, Supabase is open source with a self-hostable core, so the project is not locked into a proprietary vendor. The same schema, policies, and functions that run on the managed service can also run on infrastructure the operator controls, keeping a path to full data sovereignty open for health data that may eventually need to reside under the operator's own jurisdiction. Second, Supabase enabled a small team to provision Postgres, authentication, Row-Level Security, and Edge Functions quickly enough that the time saved went into the product rather than backend infrastructure. Third, for a first demonstration aimed at internal and clinical stakeholder feedback rather than at real patient data at scale, the managed cloud option is an acceptable interim tradeoff, provided that genuine patient data is not processed on it in this form. This is explicitly a decision made for the demonstration stage, not for final production. A production deployment would move to a self-hosted Supabase instance or to a bespoke backend, running on the operator's own infrastructure within the EU or EEA and covered by a finalized data-processing agreement scoped to real patient data. The next chapter's Outlook identifies that step as concrete future work.

4.4 Compliance Limitations of the Current Prototype

When measured against a production-grade, compliant system, several elements are missing. No Data Protection Impact Assessment has been carried out, although the sensitivity of the data would require one. There is no formal consent-capture flow beyond the act of creating an account (which is normally done by a lab member or admin), so consent is not recorded, versioned, or withdrawable in the way the regulation requires it. There is no documented data retention or deletion procedure, which leaves the storage limitation principle and the right to erasure unaddressed at

the process level. There is no audit trail beyond what Postgres and Supabase log by default, so access to health data is not independently reviewable. And the system relies on a managed third-party processor without a data-processing agreement scoped to real patient data.

5 Implementation Challenges and Design Decisions

Rather than restating what was built, this chapter examines the concrete tensions that arose during implementation and the trade-off taken in response to each. Every section pairs a constraint with the decision it forced and states what that decision left unresolved.

5.1 Cross-Platform Code Sharing

The central tension in a two-client project is that some concerns can be shared between the clients and others cannot. React Native, bundled by Metro, and the React web app, bundled by Vite, cannot cleanly share user-interface components: they render through different primitives and lay out differently, so a component written for one does not run on the other. Yet both apps must agree exactly on the data. If a column is renamed on the web side while the mobile side still uses the old name, the discrepancy goes undetected until a user reaches the affected path at runtime. The two apps, therefore, had to be aligned where it mattered and independent elsewhere.

5.2 Designing for Accessibility: Motor-Impaired and Assistive-Technology Users

The participant app was shaped throughout by the mouth-stick and limited-dexterity requirement. Navigation is built around a single Home screen of large tiles instead of a bottom tab bar, which reduces the number of small, closely packed targets one might miss. Forms such as the daily check-in present one question per screen with the input stretched to fill the available space, so that the interactive area is as large as the layout allows, and multi-step flows stay short. These layout choices follow directly from the requirement that an imprecise pointer should still succeed. The app was tested with a participant who is motor-impaired and uses a mouth stick to operate a touchscreen. This confirmed that the core flows, including completing a daily check-in, are usable with actual assistive input. The test involved a single participant and did not cover the full range of adapted inputs, such as switch access and head pointers, so it does not reflect a formal conformance evaluation.

5.3 Offline Resilience and Data Integrity

Participants use the app unsupervised and sometimes on poor connections, which creates a tension between availability and correctness. On one hand, a check-in or a diary entry should not vanish, and the app should still show a participant their own

information when the network is slow at startup. On the other hand, loosening the client's assumptions about connectivity must not permit duplicated or inconsistent records. The decision addressed the two aspects separately. For availability, TanStack Query is backed by a device-storage persister, so the last-known check-ins and visits render immediately from cache, and a cold start without a connection degrades to stale data rather than to a blank screen. Keeping usable data on the device in this way reflects the local-first principle of treating the on-device copy as primary (Kleppmann et al., 2019). For correctness, the guarantees live in the database, not the client. The one-check-in-per-day uniqueness constraint means that even a duplicated or retried submission is rejected server-side, so read-time resilience never has to compromise write-time correctness. What this deliberately does not provide is true offline write queuing and later synchronization. A participant who is fully offline can read but cannot reliably submit, because building a robust sync layer, with the conflict handling it implies, was out of scope for a first prototype. The chosen design covers the achievable subset. Resilient reads and correct writes, without the more demanding capability of offline writes, which is not yet provided.

5.4 Prototype Boundaries: Mocked and Deferred Components

A few parts of the system are mocked. Mocking them was the appropriate choice for a first demonstrable version rather than an undocumented shortcut. Each mock was scoped to match the interface of the eventual real component, so that replacing it later does not affect the code that uses it. The lab-visit file upload is parsed by a placeholder `parseVisitData`. It reads the canonical session file into the typed results object, but its signature, taking a file and returning a `VisitResults`, is the contract the real parser will honor, so a more capable ingestion routine can replace it without changes to any caller. The home-training feature is fully mocked: no real BCI hardware is available at this stage, so the mini-games simulate the chip and store connection state and level progress locally on the device. The training flow is written so that a real chip session can replace the simulation once hardware becomes available. The `sendMessage` Edge Function sits at a similar boundary. Direct participant-to-lab contact is now a first-class in-app feature backed by the `chat_messages` thread, so this function is no longer the primary means of contact. It remains as an out-of-band notification hook that currently forwards to email, with where those notifications ultimately land left as an open integration choice. In each case, these are documented, bounded pieces of deferred work with stable interfaces, not concealed gaps.

6 Conclusion

6.1 Summary

The project set out to build a monitoring tool for people recovering from a BCI implant that is accessible and compliant by design, and to be precise about how far the

first prototype progresses toward that goal. What was built is a two-client system: a participant mobile app and a lab web portal on a single shared Supabase backend, with access enforced by Row-Level Security and a single shared data-access layer that keeps the two clients consistent. The prototype demonstrated that a small team can produce a working, end-to-end system in which the important invariants reside in the database, each audience receives an interface suited to its conditions, and accessibility and compliance are designed in rather than deferred. What it leaves open is the distance between a demonstrable system and one that could safely carry real patient data, which the following sections address.

6.2 Limitations

The limitations are stated here in order of their significance for this particular patient group, drawing together the gaps already identified in the preceding chapters.

1. **Limited accessibility validation.** The participant app was designed for mouth-stick and motor-impaired use and was tested with one participant who operates a touchscreen with a mouth stick, confirming the core flows. It has not yet undergone a formal evaluation across the full range of assistive devices and users, which is the most important validation still outstanding, given the target population.
2. **Prototype-grade compliance.** The compliance posture suits a demonstration but not the processing of real patient data at scale. There is no Data Protection Impact Assessment, no formal consent flow, no audit trail beyond default logs, and a reliance on a managed third-party processor without a scoped data-processing agreement.
3. **Mocked components.** Several parts are intentionally simulated, namely the visit-file parsing, the BCI chip integration behind the training exercises, and the downstream routing of the out-of-band notification hook (in-app participant-to-lab messaging itself is implemented).

6.3 Outlook

The most important next step is a formal accessibility evaluation against WCAG 2.2 with a broader group of mouth-stick and switch-access users, extending the single-participant test already conducted to the full range of adapted inputs. In parallel, a production deployment path would move the backend to a self-hosted Supabase instance or a bespoke backend on the operator's own EU or EEA infrastructure, accompanied by a completed Data Protection Impact Assessment, consent management, audit logging, and a finalized data-processing agreement in which a managed provider is retained. The mocked components would then be replaced with real integrations as the corresponding hardware and process partners become available: genuine session-file ingestion, a real chip session feeding the training data, and a chosen destination for the out-of-band notification hook. Finally, if the system's scope expands beyond adherence monitoring to anything that informs clinical decisions, an

assessment of EU MDR applicability would be necessary, since that is the boundary at which this tool would begin to constitute a regulated medical device.

References

- Collinger, J. L., Wodlinger, B., Downey, J. E., Wang, W., Tyler-Kabara, E. C., Weber, D. J., McMorland, A. J. C., Velliste, M., Boninger, M. L., & Schwartz, A. B. (2013). High-performance neuroprosthetic control by an individual with tetraplegia. *The Lancet*, 381(9866), 557–564. [https://doi.org/10.1016/S0140-6736\(12\)61816-9](https://doi.org/10.1016/S0140-6736(12)61816-9)
- European Parliament and Council of the European Union. (2016, May 4). *Regulation (EU) 2016/679 on the protection of natural persons with regard to the processing of personal data and on the free movement of such data (General Data Protection Regulation)*. <https://eur-lex.europa.eu/eli/reg/2016/679/oj>
- European Parliament and Council of the European Union. (2017, May 5). *Regulation (EU) 2017/745 on medical devices*. <https://eur-lex.europa.eu/eli/reg/2017/745/oj>
- European Parliament and Council of the European Union. (2025, March 5). *Regulation (EU) 2025/327 on the European Health Data Space and amending Directive 2011/24/EU and Regulation (EU) 2024/2847*. <https://eur-lex.europa.eu/eli/reg/2025/327/oj>
- Free, C., Phillips, G., Galli, L., Watson, L., Felix, L., Edwards, P., Patel, V., & Haines, A. (2013). The effectiveness of mobile-health technology-based health behaviour change or disease management interventions for health care consumers: a systematic review. *Plos Medicine*, 10(1), e1001362. <https://doi.org/10.1371/journal.pmed.1001362>
- Hochberg, L. R., Bacher, D., Jarosiewicz, B., Masse, N. Y., Simeral, J. D., Vogel, J., Haddadin, S., Liu, J., Cash, S. S., Smagt, P. van der, & Donoghue, J. P. (2012). Reach and grasp by people with tetraplegia using a neurally controlled robotic arm. *Nature*, 485(7398), 372–375. <https://doi.org/10.1038/nature11076>
- Kleppmann, M., Wiggins, A., Hardenberg, P. van, & McGranaghan, M. (2019). Local-first software: you own your data, in spite of the cloud. *Proceedings of the 2019 ACM SIGPLAN International Symposium on New Ideas, New Paradigms, And Reflections on Programming and Software (Onward!)*, 154–178. <https://doi.org/10.1145/3359591.3359737>
- Wobbrock, J. O., Kane, S. K., Gajos, K. Z., Harada, S., & Froehlich, J. (2011). Ability-based design: Concept, principles and examples. *ACM Transactions on Accessible Computing*, 3(3), 1–27. <https://doi.org/10.1145/1952383.1952384>
- Wolpaw, J. R., Birbaumer, N., McFarland, D. J., Pfurtscheller, G., & Vaughan, T. M. (2002). Brain-computer interfaces for communication and control. *Clinical Neurophysiology*, 113(6), 767–791. [https://doi.org/10.1016/S1388-2457\(02\)00057-3](https://doi.org/10.1016/S1388-2457(02)00057-3)
- World Wide Web Consortium. (2023, October 5). *Web Content Accessibility Guidelines (WCAG) 2.2*. <https://www.w3.org/TR/WCAG22/>